FPGA-Based Multi-Joint Robotic Arm Control Using Verilog HDL and PWM Technique

Prajna K M[1], Vimal V P[2] and Rajesh Ranjan[3]

[1,2]Department of Electrical, Electronics and Communication Engineering,
GITAM Deemed to be University, Bengaluru-561263.

[3]Department of Mechanical Engineering,
GITAM deemed to be University, Bengaluru-561263.

Executive Summary

The purpose of the project is the design, implementation, and hardware verification of a multijoint robotic arm on an FPGA using Verilog HDL. The main goal of the project is the development of a real-time robotic arm control system capable of controlling multiple joints using Pulse Width Modulation (PWM) signals generated by an FPGA. At the initial stage of the project implementation, PWM was implemented and verified on the Nexys-4 FPGA board using LEDs. The PWM controller was connected to a DC motor and a DC geared motor using an L293D motor driver for the investigation of the dependence of motor speed and torque on duty cycle under various operating conditions.

To achieve the precise position control, further development of the project included the use of SG5010 and MG995 servo motors equipped with built-in position feedback control. The pulse widths needed to obtain the entire 180° rotation range of the servos were calculated experimentally. A multi-joint robotic arm consisting of three joints: shoulder, elbow, and wrist, was designed, where pre-programmed robotic arm poses were selected by means of FPGA switches. The sequential strategy of movement of the robotic arm was used, where the shoulder joint moved first, and then the elbow and wrist moved simultaneously to return to the default home position.

The entire system was designed, simulated, and implemented using Xilinx Vivado on the Nexys-4 FPGA board. The experimental results proved that the FPGA was able to generate PWM signals to control multijoint servos in a coordinated way, resulting in smooth robotic arm movements. This project allows one to see the benefits of using FPGAs in motor control, such as determinism, parallelism, and real-time capabilities. The project can be considered as an example of using Verilog HDL for the creation of robotic automation systems. It serves as a good basis for further work with PID control, encoders, computer vision, and IoT-enabled robotics.

Table of Content

1.INTRODUCTION

The introduction of robotic systems into the automation process of industries has proved to be inevitable due to their capabilities of performing repetitive tasks, accurately, and with very high speed. The need for robotic systems in Industries has been increasing as the trend of Industry 4.0 and smart manufacturing keeps growing day after day. Among many types of robotic systems, multi-joint robotic arms are being used extensively because of their flexibility, preciseness, and adaptability while performing the complex manipulation task such as pick and place tasks, assembling, welding, packaging, and inspection. The performance of the robotic system depends on how effectively the motor control system drives each joint of the robotic system.

The Field Programmable Gate Array (FPGA) has become an appealing hardware architecture for the development of real-time control systems owing to its inherent nature of parallel processing, deterministic behaviour, fast execution, and reconfigurable feature. Unlike typical microcontrollers that follow a sequential process of execution of code, an FPGA utilizes hardware to implement logic, which allows the simultaneous execution of several control processes. The FPGA is therefore more favourable in robotics application where several motors have to be independently controlled. In addition, HDLs like the Verilog language facilitate the development of efficient digital control systems that can be simulated prior to the actual hardware development.

The main aim of the proposed internships project was designing and experimentation of multi-joint control of robotic arm using Verilog HDL on the Nexys-4 FPGA development kit. Rather than starting the project by implementing the servomotors directly, the project was undertaken in a stepwise manner for understanding the characteristics of different motors used in robotic systems. First, PWM signals were generated and verified on the FPGA using LED indicators. It was done to check that proper duty cycle generation is obtained from the PWM signals. PWM signal generation technique is one of the most frequently used techniques for controlling the speed, power, and position of electric motors.

Having verified the functionality of PWM generator, a standard DC motor was connected to the FPGA via an L293D H-bridge motor driver. The aim of this step was to observe the effect of changes in PWM duty cycle on the speed of the motor depending on operating conditions. It was experimentally observed that with an increase in the duty cycle, the voltage applied to the motor and hence its speed increased. The DC motor proved to be very efficient in various duty cycle ranges and made an excellent subject for observing FPGA-based PWM speed control system. At this step, various duty cycle values were applied using FPGA switches, thus making the practical observation of the motor behaviour and PWM features possible.

The research project was further advanced to involve a DC geared motor to investigate the effect of mechanical gear ratio on the motor operation. In contrast to the standard DC motor, the geared motor had much higher torque but required a larger duty cycle to rotate the motor. It turned out from the practical experiments that with smaller duty cycles, the required starting torque of the geared motor could not be generated; with larger duty cycles, the motor operated stably. The practical experiments also showed that the correct choice of an external power source is very important. Indeed, the widely-used 9V battery did not provide enough current for the continuous operation of the motor. Using the regulated power source helped solve this problem.

While the speed control using PWM proves effective in the case of DC motors, it is not possible to control the angular position without using an external sensor such as an encoder. As robots need the joints to hold the angles precisely rather than rotate continuously, the last step of the project involved installing servomotors controlled by the FPGA controller. SG5010 and MG995 servomotors have been chosen due to

their internal feedback system which is made up of a potentiometer, the control circuit, and a DC motor that is put into one actuator. Internal closed feedback loop allows the servomotor to precisely hold angular positions according to the widths of PWM pulses without the need of additional external sensors in order to use in the simple robotic arms.

The experiments have been done in order to find out the pulse widths necessary to reach the whole angular range of the SG5010 and MG995 servomotors. Though the recommended standard pulse width for most servos is 1 ms to 2 ms, the experiments showed that a more extensive range of pulse widths from 0.5 ms to 2.5 ms gives almost the whole 180° mechanical rotation of the used servo motors. These values of the width of pulses were set in the FPGA controller through programming. It can be seen that practical experimenting was crucial to reach correct values.

On the basis of the calibrated servo control, a three-joint robotic arm including shoulder, elbow and wrist joints was developed. Predefined home positions and several operational poses were programmed for the robotic arm using FPGA switches. The sequential motion algorithm for the robotic arm to simulate realistic motion was used. While performing an operation, the shoulder joint rotates first, then both elbow and wrist joints rotate simultaneously, afterwards the robotic arm stays idle for a certain time period after which it automatically returns to the predetermined home pose. Thus, this motion algorithm shows the multi-joint motion coordination together with the stability of servo operation.

The whole system was developed using Verilog HDL language, simulated using Xilinx Vivado software, and implemented on the Nexys-4 FPGA development board. In order to verify the hardware implementation, the use of the external regulated power sources was required in order to ensure reliable servo operation and stable PWM generation. As part of the internship project, much attention was paid to the analysis of practical operation of various motors, their comparison in relation to the usage in robots, as well as the influence of such parameters as the PWM duty cycle, pulse width, torque, voltage and mechanical loads on the motor operation. Thus, this part of the project provided valuable information from the engineering point of view rather than just the theoretical implementation.

Overall, this internship project demonstrates the successful implementation of the real-time multi-joint robotic arm control system using FPGA technology. Digital hardware design, PWM generation, motor control, sequential motion planning, and hardware implementation of the project are integrated into one embedded robotic platform. The gained knowledge provides a solid ground for future projects that will involve the development of a closed-loop control system with PID, implementation of feedback control using encoder, integration of computer vision and IoT, as well as development of intelligent and autonomous robotic systems. The use of FPGA provides many advantages in terms of deterministic timing, parallel processing, high reliability and scalability.

2. LITERATURE SURVEY

The rapid progress of robotics has led to an increasing demand for intelligent control systems capable of performing efficient and reliable motion control. According to John J. Craig [1], a robotic manipulator is defined as an assembly of mechanical links interconnected through actuated joints allowing the controlled motion in numerous degrees of freedom. It should be noted that industrial robotic systems are a combination of mechanical components, actuators, sensors, and control algorithms designed to perform repetitive tasks with high precision and reliability. These notions have provided the theoretical base for the design of modern robotic manipulators used in manufacturing, medical robotics, space exploration, and industrial automation.

Motor control represents one of the major parts of any robotic systems where Pulse Width Modulation (PWM) technique has become very popular due to its efficiency in speed and position control. Muhammad H. Rashid [2] describes that PWM is based on changing the duty cycle at constant switching frequency that helps to reduce power losses and improve the efficiency of the system. In addition, the author notes that PWM allows smooth speed regulation of the motor and is much more efficient than traditional linear control method. In addition, Rashid discusses the importance of H-Bridge driver circuits such as L293D for connecting low-power digital devices with higher-power DC motors. These concepts provide the theoretical base for the PWM-based motor control system implemented in the present work.

Designing of the control system is crucial for stable and accurate motion of the robot. Katsuhiko Ogata [3] describes that modern control system requires the use of mathematical modelling, transfer function, stability evaluation, and feedback that are necessary to ensure proper performance of the control system during its transient and steady-state modes. In addition, the author notes that Proportional-Integral-Derivative (PID) controllers help to reduce steady-state error, increase the speed of the transient process, and improve disturbance rejection properties of industrial control systems. These notions form the theoretical base for the future FPGA-based closed-loop PID control to be implemented in the robotic arm in this work. Several works have explored the issue of the implementation of the robotic arm controllers using FPGA technology. Mansoor, Khalil, and Jasim [4] have introduced the position control system for the robotic arm based on the Pico Blaze soft-core processor implemented in Xilinx Spartan-3E FPGA. The system generated the PWM signal using software in order to control position and speed of the five DC servo motors assembled to create a robotic arm. As the authors note, the servo motors require PWM signals with the period equal to 20 ms (50 Hz), while the pulse widths in the range of 0.5 ms – 2.5 ms correspond to the angular position from 0° to 180°. The experimental implementation allowed achieving accurate positioning of servos while consuming only about 4% of the FPGA slices. However, the proposed system used an embedded soft-core processor programmed in assembly language instead of completely hardware-based implementation in Verilog HDL.

In order to enhance multi-axis robotic control, Kung *et al*. [5] have developed an FPGA-based servo control integrated chip for six-axis articulated robotic manipulator. Their research includes motion trajectory planning as well as six independent position, speed, and current controllers in the single FPGA in the System-on-a-Programmable-Chip (SoPC) architecture. In particular, the authors used the embedded NiosII soft-core processor together with special IP blocks for servo control. The experimental validation showed that FPGA implementation reduced the size of the controller, improved synchronization among multiple robotic joints, and increased computational efficiency by means of parallel processing. Thus, the authors concluded that FPGA technology is very effective in complex multi-axis robotic systems requiring realtime control.

Another contribution to the field was made by Karcı and Tangel [6]. In their paper, they designed and implemented the fully autonomous five-degrees-of-freedom mobile robotic arm using FPGA technology. The robotic system included DC motors, RC servo motors, and ultrasonic sensors that were connected to the FPGA platform. PWM signal generated using VHDL modules controlled speed and position of the motors, while FPGA parallel processing provided simultaneous working of multiple robotic systems. The implemented system successfully performed autonomous object detection and manipulation, proving the effectiveness and reliability of FPGA-based robotic control architectures.

With the increase in the complexity of robotic applications, the researchers began to investigate FPGA technology from another perspective. In their work, Wan *et al.* [7] have presented a detailed overview of FPGA-based robotic computing and described the requirements for modern robotic systems that include high computational performance, real-time and energy-efficiency constraints. They have compared FPGA platforms with traditional CPU and GPU architectures and found that FPGA implementations significantly outperform other platforms in terms of latency, energy efficiency, and massive parallel processing. These features make FPGA technology very suitable for robotic perception, localization, motion planning, and real-time motor control applications.

In their subsequent review, Wan *et al.* [8] discussed the current state, research challenges, and future perspectives of FPGA-based robotic computing. They noted that FPGA platforms possess several important advantages such as hardware reconfigurability, low power consumption, deterministic timing, high reliability, and adaptability to new evolving robotic algorithms. In addition, they identified several research areas that could be developed in the future, such as improvement of system-level integration, intelligent robotic control, adaptive architectures, and hardware acceleration of advanced robotic algorithms. Recent work by Ajel, Abdul Abbas, and Mnati [9] studied the optimization of the servo motor position and speed control using FPGA technology. Their implementation used the Altera DE1 FPGA development board together with the PID control algorithm in order to increase positioning accuracy and dynamic response. The authors proved that FPGA-based PID control provides flexible implementation of the servo control algorithms while increasing system precision and response characteristics.

Despite the significant progress achieved in FPGA-based robotic systems, there are several limitations in the current literature. Most of the papers concentrate mainly on the servo motor position control, embedded soft-core processors, or industrial robotic manipulators. Very few works are dedicated to experimental investigation of the behaviour of conventional DC motors, DC geared motors, and servo motors on the FPGA-based platform before their integration into a robotic arm. Many of the reported systems use encoder feedback, embedded processors, and sophisticated closed-loop algorithms, which increases the complexity and cost of implementation.

In order to overcome the existing limitations, the present work uses systematic FPGA-based hardware design approach implemented completely in Verilog HDL on the Nexys-4 FPGA development board. The work starts from the design and experimental verification of the PWM generator followed by the analysis of the relationship between the PWM duty cycle and motor speed using the conventional DC motor and DC geared motor with the L293D motor driver. Based on the results, SG5010 servo motors are calibrated and assembled into the three-jointed robotic arm consisting of shoulder, elbow, and wrist joints. Finally, the motion control algorithm based on the finite state machine is implemented to provide multi-joints coordination. The proposed architecture can be used as the basis for the further development of closed-loop feedback based on encoder, FPGA-based PID control, IoT connectivity, and intelligent robotic manipulation systems.

3. MOTOR CONTROL FUNDAMENTALS

3.1 Introduction to Electric Motors

Electric motors are electromechanical components that use the principles of magnetic fields interaction in converting electrical energy into mechanical energy. These components have an important role in modern engineering, automation, robotics, industry, household, electric automobiles, and manufacturing process. As regards to robotic applications, electric motors are used in generation of rotational movement that helps in moving joints of the robotic arm. The main selection criteria for motors include speed control, load capacity, torque, precision, and response time.

Several types of electric motors exist in order to meet requirements of engineering applications, such as DC motor, DC geared motor, servo motor, stepper motor, and brushless DC (BLDC) motor. Each of these motors has its own advantages and disadvantages. The speed of these motors is easily controlled by changing average voltage value that is used through Pulse Width Modulation (PWM). However, DC motors continuously rotate and cannot keep certain angular position without using other sensors such as encoder.

DC geared motors are enhanced versions of conventional DC motors, which include reduction gearbox for increasing output torque and decreasing rotational speed of motors. DC geared motors are used in

applications when more torque is needed in order to rotate load with lower speed. Generally, DC geared motors need more torque during starting process and stable power supply.

Servo motors are widely used in applications of robotic arms because of their precise position control with the help of internal feedback loop. Servo motor includes DC motor, reduction gears, potentiometer and internal controller all in one device. In comparison with conventional DC motors, servo motors rotate only till the needed angular position depending on PWM pulses. Thus, servo motors are used in applications with high requirements on the position control, such as robotic joints – shoulder, elbow, and wrist.

Three motor types including DC motor, DC geared motor, and servo motor were researched and implemented in this project with the help of FPGA based PWM controller designed in Verilog HDL. First, PWM signals generated by the Nexys-4 FPGA were used for control of speed of DC motor and DC geared motor by means of L293D motor driver. Finally, PWM controller was calibrated and was used for control of SG5010 and MG995 servo motors in multi-joint positioning system for robotic arm.

3.2 DC Motor Construction and Working Principle

A Direct Current (DC) motor shown in Fig 3.1 is the most popular kind of electric motor because of its simplicity, efficiency, affordability, and easy speed control capabilities. It changes electrical energy into mechanical rotational energy using the principle that when current passes through a wire placed in a magnetic field, it experiences a mechanical force.

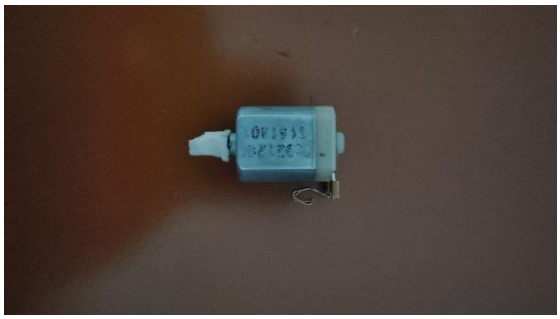

Fig 3.1: DC Motor

Components of a DC motor include the stator, rotor (armature), commutator, carbon brushes, shaft, and permanent magnets or field winding. The stator constitutes the stationary section of the motor that generates a constant magnetic field. The rotor (also called the armature) is the rotating section of the motor that has copper wires wound on its body. The commutator consists of a copper ring attached to the rotor shaft while carbon brushes allow the transfer of electrical energy between the power supply and rotating commutator.

In combination, the brushes and the commutator reverse the direction of current flowing through the armature windings in every half turn to enable continuous movement of the motor in a single direction.
The working principle of a DC motor is Lorentz Force. When a DC voltage is applied to the motor terminals, the current starts flowing through the armature windings. The interactions between the magnetic fields created by the stator and armature wires generate electromagnetic forces which produce torque to make the rotor rotate.

In this project, a DC motor was controlled using Pulse Width Modulation (PWM) provided by the Nexys4 FPGA using Verilog HDL programming. However, the FPGA could not directly supply the current needed by the motor and, therefore, an L293D H-bridge motor driver was used as the interface between the FPGA and the motor. The PWM duty cycle was varied using the FPGA switches to control the average voltage

supplied to the motor, thus changing its speed. Experimental observations indicated that an increase in the duty cycle led to an increase in the motor speed, while a decrease led to a slowing down of the motor. In summary, it was discovered that the PWM technique is an effective method of regulating the speed of a DC motor since there is no energy loss.

Unlike DC motors, they cannot accurately hold a precise angular position without additional feedback sensors like the encoders. This makes them suitable for use in applications where continuous rotation is involved, but they are unsuitable in cases where precise positioning is involved. Due to this constraint, the next step in the study will be on the DC geared motor and servo motor studies.

3.3 DC Geared Motor Construction and Working Principle

DC geared motor which is shown in Fig 3.2 is the integration of the regular DC motor and reduction gearbox into one compact construction. The gearbox is linked to the shaft of the motor and is made to reduce the speed of rotation but simultaneously increase torque. This combination makes DC geared motors suitable for the applications, where high torque and controlled motion are more preferable than speed, for instance, robotics, conveyor belt, automated guided vehicle, robotic arm, and industrial automation.

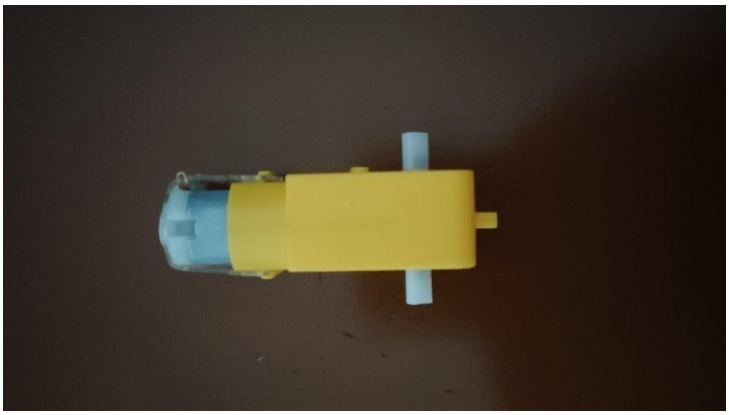

Fig 3.2: DC Geared Motor

The major components of DC geared motor are the DC motor, gearbox, gear train, output shaft, magnets, armature (rotor), commutator, and carbon brushes. Like in the case of regular DC motor, the DC motor converts the electrical energy into rotational motion. However, unlike the regular DC motor, the rotational motion in this case is transferred through the gears to the output shaft, which rotates slower with a larger torque value. Depending on the gear ratio, the speed will be reduced and torque will be increased by certain factor, for example, 50:1 gear ratio means that motor shaft rotates one fiftieth faster than the output shaft and produces fifty times higher torque excluding losses.

The principle of work of a DC geared motor is very similar to the principle of operation of a regular DC motor. When a DC voltage is applied, the current passes through the armature coils generating electromagnetic field. Interacting with the field of the motor stator, the field of the armature produces torque, thus causing the rotation of the motor shaft. This rotation is transferred through the gearbox with reduction of speed and increase of torque at the output shaft.

For the purpose of this project, a DC geared motor was connected to the Nexys-4 FPGA via the L293D HBridge motor driver. PWM signals implemented using Verilog HDL programming language were used to control the average voltage supplied to the motor. The values of different duty cycles were chosen via FPGA switches to investigate the dependence between the duty cycle and speed of the motor. During the practical testing it was discovered that the geared motor needed a much larger duty cycle in order to start rotating, compared to the regular DC motor. This behaviour is caused by the higher starting torque required

to overcome the inertia of the gearbox and the attached mechanical load. It was found out that the unstable 9V battery does not provide enough current for the proper operation of the geared motor, while regulated 6V power supply provided the consistent results.

The major advantage of the DC geared motor compared to a regular DC motor is higher torque, better loadcarrying ability, and smoother operation at low speeds. On the other hand, a geared motor is less responsive to the changes in speed and needs more stable power supply capable of providing enough current. From the experiments carried out in the course of this project, it becomes clear that while the use of PWM allows controlling the speed of the DC geared motor, special attention should be paid to the choice of duty cycle and power supply. These conclusions help to understand the practical aspects of the motor control and the advantages and disadvantages of the geared motors before proceeding with the precise position control using servo motors.

3.4 Servo Motor Construction and Working Principle

A servo motor is an electromechanical actuator having a feedback loop which ensures a precise control of angular position, velocity and torque. While a conventional DC motor rotates endlessly when powered with electricity, a servo motor turns until a specific angular position determined by an input signal and holds it using a feedback system inside. Servo motors are popularly used in robotic arms, CNC machines, stabilization systems for cameras, UAVs, industry, and biotechnology due to their small size and ability to give precise location and quick response time.

The structure of the servo motor includes a number of elements placed in one unit. A servo motor contains an inbuilt DC motor that generates rotational motion, gears to reduce the speed of rotation and increase the torque, a potentiometer to detect the angle of rotation of the output shaft, angle of rotation of the output shaft, an electronic control circuit comparing the desired position from the input control signal with the position provided by the potentiometer, an output shaft and a three-pin cable for electric power and control signals. If the control circuit finds a discrepancy between the desired and the actual position, it drives the internal DC motor until the error disappears and then stops the motor. Therefore, a servo motor is a closedloop device with high positioning precision without an external encoder.

Fig 3.3: MG995 Servo Motor

Fig 3.4: SG-5010 Servo Motor

The SG5010 shown in Fig 3.4 and MG995 shown in Fig 3.3 servo motor used in this project works on Pulse Width Modulation (PWM) signals. Contrary to the DC motors where the duty cycle mainly regulates the

speed, a servo motor decodes the pulse width to establish the target angular position. The PWM signal has the frequency of about 50 Hz which means that its period is 20 ms. In each 20 ms period the width of the HIGH pulse establishes the required angular position of the shaft. When implementing a real-life application, the SG5010 and MG995 servo motor was calibrated experimentally and found that a pulse width range of about 0.5 ms to 2.5 ms corresponds to nearly full 0° to 180° angular range. The calibration data was added into the design of the FPGA to allow precise positioning of the joints of the robotic arm.

In this project the servo motors were directly connected to the Nexys-4 FPGA without using any additional motor drivers since the FPGA just produces the PWM control signals while the servo motor has internal control circuitry and power electronics. The servo motors were powered from an external power supply having voltage 6 V, and the PWM control signals were produced using Verilog HDL on the FPGA. There were two SG5010 and one MG995 servo motors being used for the control of the shoulder, elbow and wrist joints of the robotic arm. There was a home position defined, and different robotic arm positions were selected using the FPGA switches. A sequential algorithm was implemented wherein the shoulder joint moved first and the elbow and wrist moved simultaneously before moving back to the home position. The experiments showed that there was smooth, stable and repeatable positioning of the robotic arm joints proving that servo motors are perfect for FPGA robotic arm applications.

Table 1.1: SG-5010 Specifications

Parameter	**Specification**
Model	SG5010 Servo Motor
Weight	47 g
Dimensions	40.6 × 20.5 × 38 mm
Operating Voltage	4.8 – 6.0 V DC
Stall Torque	5.5 kg·cm @ 4.8 V6.5 kg·cm @ 6.0 V
Operating Speed	0.19 s/60° @ 4.8 V0.15 s/60° @ 6.0 V
Gear Type	Nylon Gear
Operating Temperature	0°C to 55°C
Dead Band Width	1 µs
Servo Wire Length	32 cm
Servo Connector	JR Type (Compatible with JR and Futaba)
Servo Arms	Servo arms and mounting screws included

Connector Compatibility	Futaba, JR, Hitec, GWS, Cirrus, Blue Bird, Blue Arrow, Corona, Berg, Spektrum
Certifications	CE and RoHS Approved

Table 1.2: MG995 Specifications

Parameter	**Specification**
Operating Voltage	4.8 V – 6.6 V DC
Stall Torque	9.4 kg·cm @ 4.8 V11 kg·cm @ 6.0 V
Operating Speed	0.20 s/60° @ 4.8 V0.16 s/60° @ 6.0 V
Current Consumption	10 mA (Idle Current)170 mA (No-Load Current)1200 mA (Stall Current)
Gear Type	Metal Gear
Cable Length	30 cm
Connector Type	JR/Futaba Universal Connector
Operating Temperature	0°C to 55°C
Weight	55 g

3.5 Comparison of DC, Geared DC and Servo Motors

A servo motor is defined as a closed-loop electro-mechanical actuator intended for precise control of position, velocity, and torque. Unlike normal DC motors that rotate continuously when electricity is fed to them, a servo motor rotates only up to certain angular position depending on control signal input and holds the position through feedback. Thanks to high precision of positioning, fast reaction, and small size, servo motors are widely used in robotic arms, CNC machines, camera stabilizers, UAVs, industrial automation, and medical devices.

A typical servo motor is constructed as a set of integrated components installed in a single housing. They include DC motor, gears, potentiometer, control electronics, output shaft, and three-wire connection. DC motor generates rotational movement; gear system slows down the motor rotation and increases output torque. Potentiometer is attached to the output shaft and provides continuous measurement of its angular position. Control electronics compares the commanded position via control signal with the measured one via potentiometer. In case there is any discrepancy, controller drives internal DC motor and makes position error equal to zero and then stops the motor to hold commanded position. Closed loop control allows reaching high accuracy of positioning of a servo motor without using external encoder.

Used in this project SG5010 and MG995 servo motor works by receiving the PWM control signals. Unlike DC motors where the duty cycle controls the motor speed, servo motor interprets the pulse width of PWM signal as required position of its shaft. Frequency of PWM signal is around 50 Hz and hence period is 20 ms. For each 20 ms period of PWM signal, duration of the HIGH pulse determines the required angular position of the shaft of servo motor. During practical use of SG5010 and MG995 servo motor, it was experimentally determined that pulse width variation from 0.5 ms to 2.5 ms resulted in almost full rotation 0° to 180° of the shaft of servo motor. This calibration was applied in the FPGA design to allow correct positioning of robotic arm joints.

In this project, servo motors were directly connected to the Nexys-4 FPGA without any external driver because FPGA only supplies the control signal (PWM) while the servo motor has its own control circuitry and power electronics inside. Servo motors were powered by external 6V supply source while PWM signals were generated in FPGA using Verilog HDL code. Two SG5010 and one MG995 servo motors were used to control shoulder, elbow and wrist joints of robotic arm. Home position of robotic arm was set beforehand and different poses of robotic arm were chosen via FPGA switches. Sequential control algorithm was implemented when shoulder joint moves first and then elbow and wrist joints move simultaneously to reach new pose and then automatic return back to home position occurs. Experimental results showed smooth and repeatable positioning which proves that servo motors are highly suitable for FPGA-based robotic arm applications.

3.6 Applications in Robotics

Robotics is considered one of the most rapidly developing areas in engineering today and is utilized in such sectors as manufacturing, healthcare, agriculture, defence, logistics, space exploration, and domestic use. The heart of any robotic system is an efficient motion control device allowing the robot to conduct certain motions. For generating movements electric motors such as DC motor, DC geared motor, and servo motors are commonly used whereas controller such as microcontroller or Field Programmable Gate Array (FPGA) provides intelligence that regulates motors operation. Motor choice should be done depending on the particular application and its requirements including speed, torque, accuracy, loading capacity, and time response.

The first area where robotics can be used is industrial automation where robots execute repetitive actions such as assembly, welding, painting, material handling, packaging, and testing. Such work requires precision motion that is why it is more preferable to use servo motors for controlling robotic joints due to their internal feedback system. Servo motors allow obtaining precise angular positioning. FPGA allows achieving deterministic timing and parallel processing and therefore controlling several robotic joints at once.

When it comes to mobile robotics, DC motors and DC geared motors are used to control wheels and locomotion system. Commonly DC motors are used in cases when high speed is required while DC geared motors are used in case when higher torque is necessary to move heavy robots or to overcome difficult terrain. It is possible to change speed of robot's movement with PWM-based speed control and not losing much energy in this process. That is why this kind of motors is appropriate for autonomous vehicles, warehouse robots, and AGV (Automated Guided Vehicle). In practice usually robotic systems include geared motors since they have better load-handling capability and lower speed operation.

Robotics can also be implemented in medicine and healthcare sphere as robots assisting in surgery, rehabilitation exercises and automating lab works. Robotic surgical arms require extremely high accuracy and smooth motion control that is obtained with the help of servo motors controlled by embedded systems.

Likewise, robotic prosthetic limb and rehabilitation devices use servo motors for mimicking natural movement of human joints. There is another area called agricultural robotics where robots perform such tasks as crop monitoring, precision spraying, harvesting and weed removal. These robots use both DC geared motors to move and servo motors to control robotic arms or spraying mechanism.

For the purpose of this project different motor types were practically investigated using FPGA. Initially, PWM generated signals from the Nexys-4 FPGA were used to investigate speed control of DC motors and DC geared motors using L293D motor driver. Eventually, SG5010 servo motors were used to control shoulder, elbow, and wrist joints of robotic arm.

4. PWM-BASED MOTOR CONTROL

4.1 Pulse Width Modulation (PWM)

Pulse width modulation is one of the most common techniques used in digital control for regulating the supply of electrical power to electronic gadgets and electric motors. The method of pulse width modulation works by switching the power supply between ON and OFF states rather than varying the value of the supply voltage. The percentage of the HIGH signal during one cycle is called the duty cycle. PWM is an energy-efficient way of motor control because it allows for changing the average supply voltage and thus controlling the amount of power without any losses.

Two key characteristics of the PWM signal include frequency and duty cycle. Frequency shows how many PWM cycles take place per second while the duty cycle is expressed in percentage and shows what part of time the signal is HIGH during one cycle. For example, the 50% duty cycle implies that the signal stays high for half the time of the period and low for the other half.. Higher duty cycle implies higher average voltage supplied to the motor, therefore the motor will rotate faster, and vice versa. As the PWM signal switching takes place with a high frequency, the motor reacts to the average voltage only.

In this internship project, PWM signals were created using Verilog HDL on the Nexys-4 FPGA development board. At the initial stage of experiments, PWM was verified by connecting the PWM signal to LEDs and observing the changes in the light intensity depending on duty cycle. Then the same PWM signals were applied to a DC motor and DC geared motor using L293D H-Bridge motor driver for investigating the connection between duty cycle and motor speed. In further studies, PWM method was used for the control of SG5010 servo motors when the pulse width of the 50 Hz PWM signal was used for defining the angular position of the servo motor instead of its speed. The experimental calibration was performed to get pulse widths necessary for covering the entire range of the angle positions (from 0° to 180°).

Thus, it became clear that the use of PWM allowed for controlling different types of motors using one FPGA platform. For the DC and DC geared motors, PWM allowed for the effective regulation of the average supply voltage and speed control; for the servo motors, the PWM technology allowed for getting precise pulse widths and for achieving the exact angular position of the robotic arm joints. The successful implementation of PWM using FPGA proved some advantages of the technology: high precision, deterministic timing, low power consumption and possibility of creation of several independent PWM channels at the same time.

4.2 Duty Cycle Concept

The duty cycle is among the most significant parameters of a Pulse Width Modulation signal and refers to the percentage of time within one PWM signal period during which the signal is HIGH (ON). The duty cycle controls the average power delivered to the load when the PWM frequency remains constant. In cases where the duty cycle is 0%, the output is always off. Duty cycles of 100%, on the other hand, imply outputs that are always ON. The varying intermediate duty cycle values allow for different average voltages being provided to the motor and hence help in controlling the performance of the motor efficiently without altering the supply voltage. The mathematical formula for calculating the duty cycle is given by:

$$\text{Duty Cycle (\%)} = \frac{\text{ON Time}}{\text{Total Time Period}} \times 100 \quad (4.1)$$

In this project, the duty cycle was created by programming Verilog HDL code on the Nexys-4 FPGA board and then varied through the use of FPGA switches. This was done in order to analyse the effect of the duty cycle on the performance of the motors. The DC motor and DC geared motor were subjected to various duty cycle levels using an L293D motor driver and the results from the experiment revealed that increase in the duty cycle led to an increase in the average voltage supplied to the motor hence increasing the motor's speed. However, in the case of the SG5010 servo motor, the PWM frequency was constant at 50 Hz.

4.3 Relationship Between Duty Cycle and Motor Speed

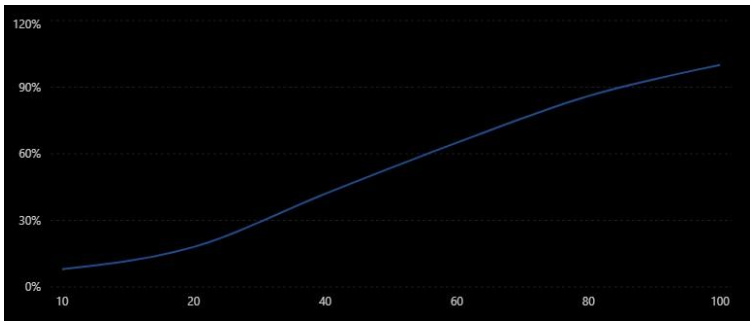

Fig 4.1: Duty Cycle vs Motor Speed Analysis Graph

The velocity of a DC motor is directly proportional to the duty cycle of the Pulse Width Modulation signal. An increased duty cycle increases the average voltage received by the motor since the time for which the power supply is ON during one PWM cycle is more. As a result of an increased average voltage, there will be increased speed in the motor as shown in Fig 4.1. Similarly, if the duty cycle is reduced, there is a reduction in the average
voltage of the supply, making the motor run slowly. The PWM is a very effective system of controlling the speed of the motor as it adjusts the average power received by the motor without changing the power voltage and hence saving power.

In this experiment, PWM waveforms were created in Verilog HDL to be used to control the speed of a DC motor and a DC geared motor using an L293D H-Bridge motor driver. It was observed that by increasing the duty cycle, there was an increase in speed in both cases. However, the DC geared motor needed a higher minimum duty cycle to start since it has a higher torque. The SG5010 servo motor, used for the latter part

of this experiment, had no relationship between the speed and duty cycle but rather used the pulse width during a fixed 20ms time to determine its angular position.

4.4 PWM Generation Using FPGA

PWM signal in this project was obtained using the Nexys-4 FPGA development board with the programming language Verilog HDL. Nexys-4 FPGA runs at a system clock rate of 100 MHz, which was taken as the reference clock for generation of the PWM signals. Counter module was developed in Verilog to continuously count the clock pulses. A comparator then compared the count value with the preselected duty cycle value using the FPGA switches. When the count value is less than the duty cycle value, the output of the PWM remains HIGH, and when the count value is greater than the duty cycle value, the output of the PWM becomes LOW. With this hardware approach, PWM signals with different duty cycles but at the same fixed frequency could be generated.

Prior to connecting the motors to the PWM generator, experiments were conducted on the PWM generator using the onboard LEDs of the Nexys-4 FPGA board. In the process, the intensity of illumination from the LEDs depended on the duty cycle values programmed. Lower duty cycles would result in dimming light since the LED would remain ON for less amount of time, while higher duty cycles would result in bright illumination since the LED would remain ON for more amount of time. This experiment confirmed that the PWM generator was working as expected.

Once the PWM generator passed all tests, it was later connected to the L293D H-Bridge motor driver for controlling both DC motor and DC geared motor. Finally, the PWM generator was changed into one that would generate 50 Hz control signals for the SG5010 servo motor. Thus, using this FPGA board, different PWM signals could be generated for different applications.

4.5 PWM Frequency Analysis

In addition to its shape, the frequency of a PWM signal is another critical characteristic affecting the efficiency of motor operation and speed. In this work, the PWM signal was generated by using the internal 100 MHz clock generator installed on the Nexys-4 FPGA board. To obtain the PWM signal for DC motors, the initial 8-bit counter was used; as a result, the obtained frequency of PWM signal was around 390 kHz. Such a high frequency of a PWM signal implies rapid switching voltage pulses that will be effectively averaged owing to the features of both electrical and mechanical aspects of the motor. In consequence, the motor reacts not on the actual switching pulses but on the average voltage value.

Unlike the DC motors, the servo motor requires the frequency of about 50 Hz that corresponds to the period of about 20 ms. The FPGA varies not the PWM frequency but the pulse width in order to set the required angle of movement. As a result of experimentally performed calibration for the SG5010 servo motor, it was found that pulse widths ranging from about 0.5 ms to 2.5 ms were enough to achieve nearly the full rotation from 0° to 180°.

The analysis carried out in this work shows the obvious difference in the characteristics of PWM signals for various types of motors. In particular, DC motor and DC geared motor use mainly the duty cycle for speed regulation, while the servo motor uses the pulse width to define the angle of movement. Thus, the proper selection of PWM frequency was the critical issue.

4.6 Experimental Analysis of DC Motor

In the initial phase of experimentation, a typical DC motor was interfaced to the Nexys-4 FPGA employing an L293D H-Bridge motor driver. The output pulses generated from the FPGA were delivered to the enable input pin of the motor driver whereas the direction of motor was controlled using FPGA output pins. Various duty cycles were chosen using FPGA switches in order to study their effects on the speed of DC motor. This basic experiment became the basis of future research regarding actuator control with the help of FPGA.

The observations during experimentation revealed that the speed of DC motor increases as the duty cycle value is increased. The motor rotates slowly with low duty cycle as the average voltage being given to the motor is low. Increase in the duty cycle value causes increase in average voltage level which consequently increases the speed of motor. The motor responds quickly to any change in the duty cycle. The results proved that PWM is an efficient speed control method because the average voltage can be changed without dissipating much power.

Thus, the experiments with DC motor proved that the FPGA can generate reliable and accurate PWM pulses for controlling the motor speed. It was noticed that a DC motor keeps rotating continuously even with application of power and cannot be made stationary with a certain angle without providing position feedback like encoder. Thus, DC motors are ideal for applications involving continuous motion but not suitable for robotic joints. These findings led to the experiments with geared DC motors and servo motors.

4.7 Experimental Analysis of DC Geared Motor

Following the successful integration of the DC motor, a DC geared motor was then interfaced with the FPGA through the use of the same PWM controller and L293D motor driver. The goal of this experiment is to determine how the reduction gearbox affects the performance of the motor, as well as its torque and speed characteristics. Just like in the previous experiment, different duty cycles for the PWM will be selected from the FPGA switches and the behaviour of the motor will be studied.

The results of the experiment show that the DC geared motor needed a higher duty cycle compared to the ordinary DC motor in order to start rotating. The main reason for this behaviour is that the motor requires a higher starting torque in order to start rotating because of the gearbox and output shaft inertial forces. Moreover, it was noticed that the motor operates smoother at lower rotational speeds and produces much higher torque. At first stage of the experiments, when a 9 V battery was used as a power source, there were not enough currents to make the motor operate properly. When a regulated external power supply of 6 V was used, the motor had sufficient current to operate and rotate properly during the whole experiment.

Experiments with geared motor have shown some practical differences between the speed and torque characteristics of the motor. Even though the reduction gearbox decreases the speed of the motor, it increases its torque output and makes the motor usable for such robotic applications as wheels, lifting mechanisms and carrying loads. Thus, it can be concluded that the PWM method is still applicable to speed regulation, but choosing proper power source and duty cycle is also very important in this case. Practical experience gained during these experiments made a good base for implementing the next project with a servo motor.

5. FPGA-BASED ROBOTIC ARM CONTROL SYSTEM

5.1 Nexys-4 FPGA Board

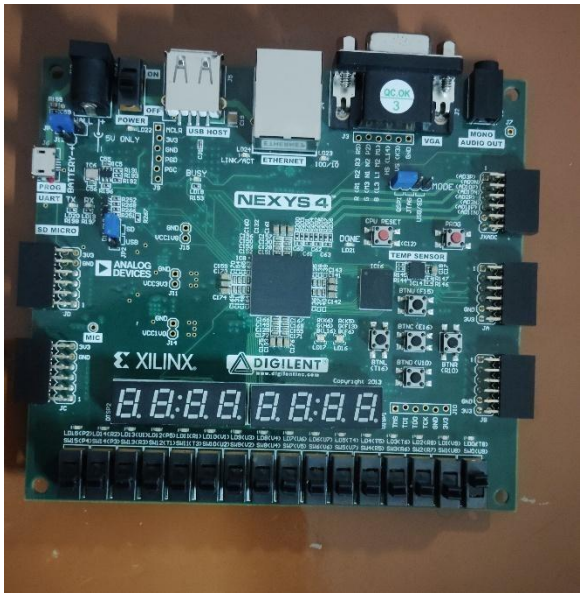

Fig 5.1: Xilinx's Nexys-4 FPGA Board

In this project, the Nexys-4 FPGA Development Board created by Digilent shown in Fig 5.1 was chosen as the platform to implement the robotic arm controller. This board contains the FPGA Xilinx Artix-7 XC7A100T chip which has hundreds of thousands of programmable logic cells, many I/O pins, memory units, and clock management unit. In contrast to microcontrollers which can process only one command at a time and do not have many processing units, FPGA allows users to create custom circuits consisting of many processing units and capable of executing multiple tasks simultaneously.

The main benefit of the Nexys-4 board is the availability of many on-board peripherals which can help to develop and test the hardware design. For example, the board has sixteen user switches, sixteen LEDs, push buttons, seven-segment displays, expansion headers, USB programming capabilities, and 100 MHz stable system clock. In the initial phase of this project, the LEDs built into the board were used to verify the functionality of PWM generators implemented in the FPGA. By changing the value of the duty cycle through the switches, the change in the brightness of LEDs proved that the PWM signals were successfully generated before interfacing the motors.

During the next phase of the work, the GPIO expansion headers which are present on the Nexys-4 board were used to connect external hardware components like L293D motor driver, DC motor, DC geared motor, and servo motors.

The outputs of the FPGA generate logic signals (up to 3.3 V), but they are able to source only a small amount of current. Therefore, some additional circuitry had to be used when it was necessary to drive the motors or power supplies for generating higher voltages. L293D motor driver was used for controlling DC and DC geared motors; servos received control PWM signals from the FPGA directly and worked from the separate 6V power supply. Adequate grounding between FPGA and external hardware helped to provide reliable data transmission during the experiments.

Nexys-4 FPGA board was proven to be a good platform to implement the complete robotic arm controller due to its fast-processing speed, deterministic timing, and possibility of hardware design. All PWM generators, control logic, finite state machines, and motion control algorithms were implemented in Verilog

HDL language and programmed into the FPGA using Xilinx Vivado Design Suite. Successful hardware implementation proves that FPGA technology is well suited for the real-time robotic application.

5.2 Verilog HDL Design Methodology

In accordance with the requirements of the project, the complete robotic arm controller has been created utilizing the Verilog Hardware Description Language (HDL). Verilog is an extensively used hardware description language which allows implementing the digital system at various levels of abstractions such as behavioural, dataflow and structural modelling. In contrast to software programming languages, Verilog does not describe algorithms but describes the architecture itself enabling the FPGA to create specific digital circuit rather than execute any instructions sequentially. Therefore, due to deterministic execution, fast processing and true parallelism Verilog is especially appropriate for motor control and robotics.

A modular design approach has been used in order to simplify development, testing and potential scalability of the designed FPGA-based controller. Instead of creating the whole robot arm controller as one module several independent functional blocks have been developed and integrated into the system. These include PWM generator, duty cycle controller, servo PWM, FSM (finite state machine), switch interface, and motion control block. Each module has been developed and simulated separately before being integrated into the complete system.

The design process has followed the classical FPGA development flow based on the Xilinx Vivado Design Suite. Firstly, Verilog code has been written for each hardware module. Afterwards, testbenches were created in order to verify functionality of each module via simulation. Simulation waveforms have been carefully analysed in order to make sure that PWM generation, duty cycle selecting, states transitions and motor control signals are implemented correctly. Then, after passing all the simulations, XDC files containing information about FPGA pin connections were created for switches, push buttons, LEDs and motor connections. After that, synthesis, implementation and hardware verification steps have been performed.

Thanks to Verilog HDL implementation of reliable and efficient FPGA-based robotic arm controller able to generate multiple PWM signals simultaneously and maintain proper timing for each motor has been achieved. All along the course of the project Verilog modules were used to control speed of the DC and DC geared motors using the L293D driver and positions of three SG5010 servo motors forming shoulder, elbow and wrist joints of the robotic arm.

5.3 PWM Generator Design

The Pulse Width Modulation (PWM) generator is one of the most important modules developed in this project, as it forms the foundation for controlling all the motors used during the internship. The PWM generator was designed using **Verilog HDL** and implemented on the **Nexys-4 FPGA**. Its primary function is to generate digital pulses with a fixed frequency while allowing the duty cycle or pulse width to be varied according to the application requirements. Initially, the PWM generator was developed for speed control of the DC motor and DC geared motor, and later modified to generate servo-compatible PWM signals for accurate position control of the robotic arm joints.

The PWM generator was implemented using a **digital counter** and a **comparator circuit**. The counter continuously counts clock pulses generated by the onboard **100 MHz system clock** of the FPGA. A duty cycle value is simultaneously selected using the FPGA switches. The comparator continuously compares the counter value with the selected duty cycle value. Whenever the counter value is less than the duty cycle value, the PWM output remains HIGH; otherwise, it becomes LOW until the next counting cycle begins. This process is repeated continuously, producing a stable PWM waveform with the desired duty cycle. Since the entire circuit is implemented in FPGA hardware, the generated PWM signal is highly accurate and free from software execution delays.

The PWM generator was first verified using the onboard LEDs of the Nexys-4 FPGA. Different switch combinations were used to generate multiple duty cycles, and the resulting change in LED brightness confirmed the correct operation of the PWM generator. After successful verification, the PWM output was connected to the **L293D motor driver** to control the speed of the DC motor and DC geared motor. For servo motor implementation, the same PWM generator was modified to operate with a **20 ms period (50 Hz)**, while varying the pulse width between approximately **0.5 ms and 2.5 ms** to achieve accurate angular positioning. Experimental calibration of the pulse widths ensured smooth and repeatable servo movement over nearly the complete 180° range.

The modular PWM generator developed during this project proved to be flexible and reusable for different motor control applications. By modifying only the timing parameters, the same Verilog module was successfully used for both speed control and position control. The hardware implementation demonstrated the advantages of FPGA-based PWM generation, including deterministic timing, high precision, low resource utilization, and the ability to generate multiple independent PWM channels simultaneously. This module became the core building block for the complete FPGA-based robotic arm control system developed during the internship

5.4 L293D Motor Driver Interface (DC Motors)

In this work, the L293D motor driver which is shown in Fig 5.2 was employed to connect the FPGA with the DC motor and DC geared motor. As the output of the Nexys-4 FPGA operates on 3.3 V logic levels and provides no more than several milliamperes of current, it cannot be connected directly to the electric motor as it requires much higher voltage and current level. The L293D motor driver acts as an intermediate power interface taking digital control signals from the FPGA and supplying necessary voltage and current from an external power source to run the motors.

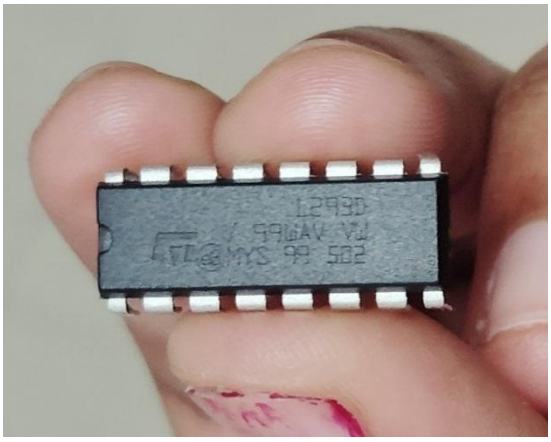

Fig 5.2: L293D Integrated Circuit

The L293D is a dual H-Bridge motor driver IC that is able to control the direction and speed of two DC motors independently. H-Bridge contains four switching elements in the bridge form; it allows reversing the polarity of the voltage supplied to the motor terminals. In the course of the project implementation, the FPGA produces one PWM signal for the motor speed control and two direction signals for the motors' forward and reverse rotation. The PWM signal is applied to the Enable (EN) pin of the L293D, and two direction signals are applied to the IN1 and IN2 input pins. The motor terminals are connected to the OUT1 and OUT2 pins, and a regulated external 6 V power supply is used as a motor power source. Common ground connection of the FPGA, motor driver and power supply is established.

Image 7

The experimental part of the work included running of both the DC motor and the DC geared motor through the L293D from FPGA-controlled PWM signals. Various duty cycle values selected with the help of FPGA switches result in corresponding motor speed. Motor direction is controlled by the change of the input pins' logic levels resulting in both forward and reverse motor rotation. It was observed that the DC motor reacts quite quickly even at rather small duty cycle values, while the DC geared motor requires higher duty cycle due to reduction gearbox overcoming. Also, it was found that a regulated external power supply provides more stable functioning of the motor comparing with the standard 9 V battery, especially at the motor start up.

As a result of the work, the L293D motor driver proved to be a reliable and efficient interface between the FPGA and the motors. The L293D provided isolation of low-power FPGA circuits from high current motor load and bidirectional motor control by means of simple digital signals. Successful implementation of the L293D interface allowed conducting the research on PWM-based motor speed control, motor direction control, and duty cycle analysis before the next stage of project – the use of the servo motor-based robotic arm control.

5.5 Servo Motor Interface

Finally, SG5010 servo motors were interfaced to the Nexys-4 FPGA in order to achieve exact position control of a three-joint robotic arm. As opposed to DC motors, a servo motor has its own internal control circuitry, gear reducer and feedback potentiometer which allows achieving closed-loop position control. Due to this control electronics, the FPGA has to deliver only a precisely controlled PWM control signal to the servo motor and the internal electronics will automatically position the output shaft to the needed angle. As a result, control of the servo motor is much easier than control of a DC motor and requires an additional H-Bridge motor driver.

SG5010 servo motor has three contacts: VCC, GND, and Signal. For this project, signal contacts of each servo motor were connected directly to separate GPIO pins of the Nexys-4 FPGA via the JA expansion header. The FPGA generates only low-power 3.3V PWM signal, thus, there is no need for voltage transformation and it could be applied directly to the servo motor control signal contact. However, due to low power requirements of the FPGA, an external regulated 6V power supply was used to drive the power consumption of the servo motors. In addition, a common ground was connected between the FPGA and the external power supply to ensure the correct work of the PWM signal control.

Three SG5010 servo motors were used to simulate the shoulder, elbow, and wrist joints of the robotic arm. During the hardware implementation process, the experimental calibration of the servo motors was conducted to obtain pulse widths for different angles of rotation. It was found out that pulse width varies from about 0.5ms to 2.5ms within 20ms of PWM period which results in almost complete rotation (from 0° to 180°) of the servo shaft. Calibrated pulse widths were implemented in the Verilog design in order to achieve correct positioning of each joint of the robotic arm. Simultaneously, three independent PWM signals for all servos were generated.

Servo motor interface developed for this project provides reliable communication between the FPGA and the robotic arm. Unlike the DC motor implementation, the external motor driver is not needed since the servo motor contains all control electronics inside itself. Experimental tests show that servo motors respond correctly to the PWM signals delivered by the FPGA and ensure smooth joint movement and stable position holding.

5.6 Switch-Based Motion Control

In order to provide a simple way of controlling the robotic arm, a switch-based motion control system was developed using the digital switches available on the Nexys-4 FPGA development board. Switches served as digital input devices allowing users to select different poses of the robotic arm without additional

interfaces and modification of the FPGA code. Each combination of switches corresponded to a predefined set of angles of the shoulder, elbow, and wrist joints and provided an opportunity to control multiple robotic arm movements by means of simple digital signals.

The digital inputs were constantly monitored by the FPGA through Verilog code. When a combination of the switches changed, the control logic selected pulse width values for each of the servo motors. The initial angles of the robotic arm were defined to be 0° for the shoulder, 30° for the elbow and 15° for the wrist, which corresponded to the home position of the robotic arm. Such angles were chosen to provide comfortable positioning of the robotic arm and enough room for its movements. Depending on the current switch value, new pulse widths were calculated for the servo motors and the robotic arm moved to the desired position.

For obtaining realistic motion of the robotic arm, a sequential control scheme was applied instead of moving all joints simultaneously. Upon selecting the new pose, the shoulder joint of the robotic arm moved first and then after some time passed, the elbow and wrist joints were activated simultaneously. After reaching the target pose, the robotic arm remained stationary for a certain period of time before returning to the home position. Sequential motion of the robotic arm joints is similar to the one used in industrial robots, where the joint movements improve stability of the robotic arm and reduce mechanical strain on the arm itself.

Thus, the proposed switch-based motion control system demonstrated how multiple poses of the robotic arm could be implemented with the help of simple hardware logic. The method allows not using any communication protocols and providing reliable and repeatable control of all three joints. Experimental results have shown that each selected switch value provides the expected movement of the robotic arm. The combination of pulse width modulation generation, servo control and simple digital logic allowed implementing a rather efficient system for controlling the robotic arm that can be developed further in future work.

5.7 State Machine Design

The FSM was implemented to synchronize movements of the three servo motors controlling the movements of the robotic arm in this project. Rather than sending commands to all joints simultaneously, this approach breaks down the whole process into several consecutive states. This makes motion smoother and prevents mechanical loading on the joint. Moreover, the algorithm is similar to one used in industrial robotic manipulators. The FSM was designed in Verilog HDL and implemented directly in FPGA without software.

The FSM implemented in this project has five major states of operation, namely Home, Shoulder Movement, Elbow and Wrist Movement, Hold, and Return Home. In the Home state, the robot is put in its resting position when the shoulder is at 0°, the elbow is at 30°, and the wrist is at 15°. When the switch input becomes valid, the controller switches from home state to Shoulder Movement. First, the shoulder servo moves to the required position depending on the chosen pose of the robotic arm. The controller then moves to the next state after certain time delay which is provided by the FPGA counter.

In the Elbow and Wrist Movement state, the elbow and wrist servo motors move to their target positions simultaneously, while the shoulder remains fixed. When the all joints arrive at the required positions, the robotic arm enters the Hold state and stops for some time to demonstrate the stability of the pose. After that, the robotic arm enters the Return Home state, during which the controller sends commands to all servo motors to move to their initial calibrated positions. When this state is finished, the controller automatically switches back to the home state waiting for new switch input.

The implementation of the finite state machine greatly enhanced the functionality of the robotic arm because all the transitions are executed by FPGA hardware, and, therefore, become deterministic and are not affected by any software related factors. Also, the FSM architecture allows adding new motion sequences or even

implementing closed loop controller relatively easily. Overall, the FSM can be considered the main controller of the robotic arm**.**

5.8 Multi-Joint Motion Control Algorithm

The primary goal of developing the motion control algorithm in this project is to coordinate the motion of several joints of the robotic arm with FPGA generated PWM pulses. In contrast to controlling a single servo motor, the motion coordination in the case of a multi-jointed robotic arm involves precise synchronization of the motion of the shoulder, elbow and wrist joints. The entire control algorithm was written in Verilog HDL and was fully executed on the Nexys-4 FPGA without any need of a processor and an external controller. The pulse width associated with each joint angle was calculated by the algorithm and used for generating independent PWM pulses for each servo motor at a time.

The robotic arm was initialized to the home position including the shoulder joint at 0 degrees, the elbow joint at 30 degrees and the wrist joint at 15 degrees. These angles were experimentally chosen to provide a stable rest position for the robotic arm with a sufficient range of angles for further manipulations. Several poses of the robotic arm were assigned to the inputs of the FPGA switches, where each switch input corresponds to a set of angles for the shoulder, elbow and wrist joints. After detecting an input from the switch, the control algorithm reads the pulse width values from the lookup table and starts the sequence of the movement commands.

To provide more stability to the robotic arm, the sequential movement approach was used in the algorithm. First, the shoulder joint is moved which provides a solid support for other joints. Then, after a certain programmable delay, both the elbow and the wrist joints start moving to their target angles. The robotic arm stays still for some time at each target position before it moves back to the home position. This approach eliminates sudden movements, decreases mechanical strain on the servo motors and results in better operation of the robotic arm. The usage of FPGA-based timing delays guarantees consistent timing of each movement during each execution of the control algorithm.

Experimental testing proved that the multi-joint motion control algorithm coordinated the operation of all three servo motors with accurate angular positioning and consistent performance. The FPGA generated independent PWM signals for all three servos without any observable timing problems, which proves the advantages of hardware-based parallelism compared to conventional sequential controls. Furthermore, the control algorithm provides a base for future improvements like trajectory planning, inverse kinematics, feedback control, object detection and even Internet-of-Things (IoT) based remote control. Therefore, the developed motion control algorithm may be considered as a core intelligence for the FPGA-based robotic arm.

6. HARDWARE IMPLEMENTATION

Table 6.1: Hardware Components Used

Component	Specification	Purpose
Nexys-4 FPGA	Artix-7 XC7A100T	Main controller
DC Motor	6V DC Motor	Speed analysis

DC Geared Motor	9V Geared Motor	Torque analysis
Servo Motor	SG5010	Robotic arm joints
Motor Driver	L293D	DC motor interface
External Power Supply	6V Regulated	Motor supply
Breadboard	Standard	Circuit assembly
Jumper Wires	Male-Male	Connections
USB Cable	USB-JTAG	FPGA Programming

FPGA Features Used

- 100 MHz onboard clock
- 16 User switches
- Push button (BTN_C)
- JA Expansion Header
- USB Programming
- 3.3V GPIO Outputs

Table 6.2: Software used to simulate

Software	**Purpose**
Xilinx Vivado	Design & Simulation
Verilog HDL	Hardware Design
XDC Constraints	Pin Assignment

6.2 DC Motor Experimental Setup

Objective

To study the relationship between PWM duty cycle and DC motor speed.

Hardware Used

- Nexys-4 FPGA
- L293D Motor Driver
- DC Motor
- 6V External Supply
- Breadboard

Experimental Setup

FPGA

↓

PWM Signal

↓

L293D Driver

↓

DC Motor

Procedure

1. Generate PWM using FPGA.
2. Select duty cycle using switches.
3. Apply PWM to L293D.
4. Observe motor speed.
5. Record observations.

Table 6.3: Experimental Observations

Duty Cycle	Motor Speed
Low	Slow
Medium	Moderate
High	Fast

Observation

- Smooth speed variation observed.
- PWM efficiently controlled motor speed.

- Stable operation achieved.

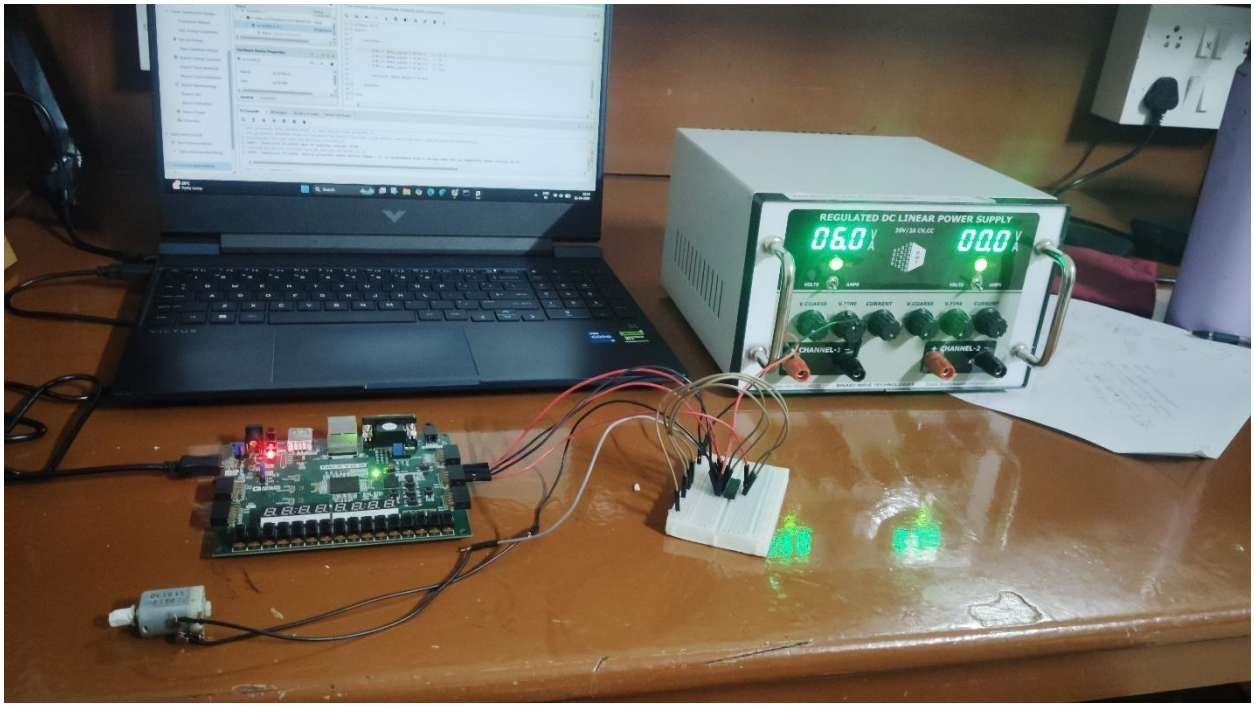

Fig 6.1: DC Motor set up for complete working of the motor

6.3 DC Geared Motor Experimental Setup

Objective
To study the effect of PWM duty cycle on a geared DC motor.

Hardware Used
- Nexys-4 FPGA
- L293D Driver
- DC Geared Motor
- 6V Regulated Supply

Experimental Procedure
- Generate PWM.
- Apply different duty cycles.
- Observe starting torque.
- Compare with DC motor.

Table 6.4: Experimental Results

Parameter	Observation
Starting Duty Cycle	Higher than DC Motor
Speed	Lower

Torque	Higher
Stability	Better

Important Observations

✓ Higher torque

✓ Smooth low-speed rotation

✓ Required regulated supply

✓ Better load handling

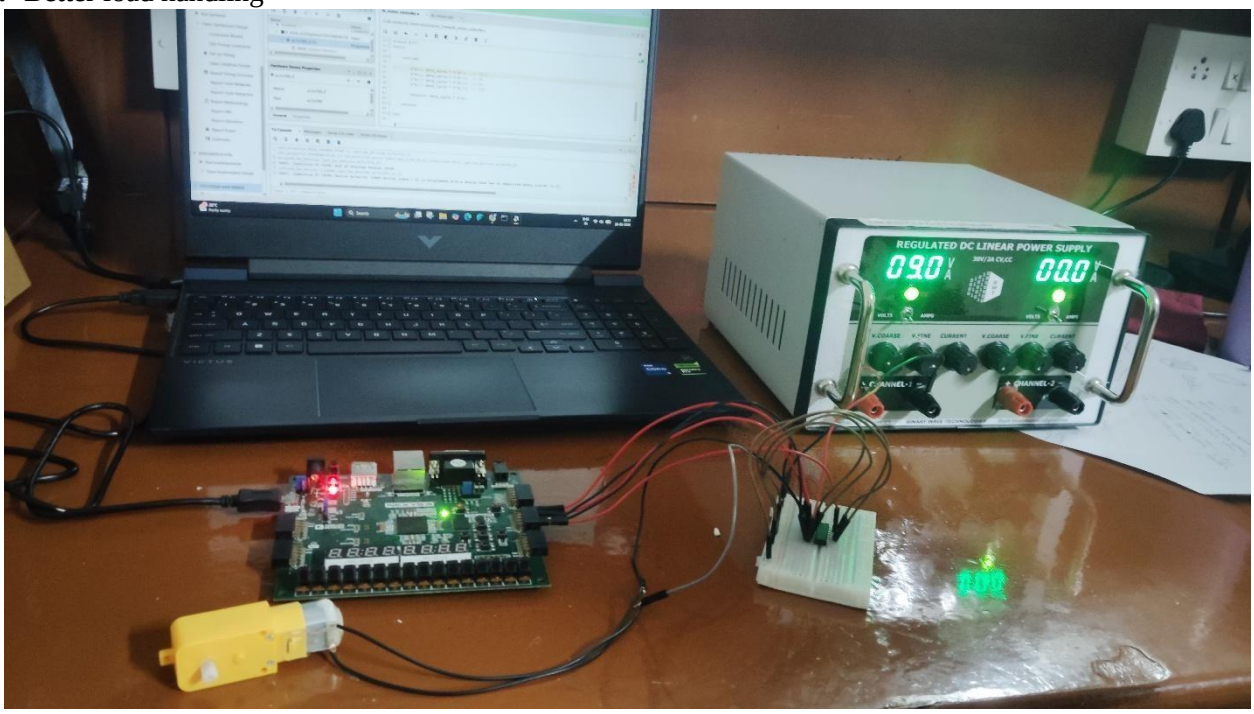

Fig 6.2: DC Gered Motor set up for complete working of the motor

6.4 SG5010 Servo Motor Experimental Setup

Objective

To implement FPGA-based position control.

Hardware Used

- Nexys-4 FPGA
- 3 × SG5010 Servo Motors
- External 6V Supply

Table 6.5: Servo Assignment

Joint	Servo
Shoulder	Servo 1
Elbow	Servo 2
Wrist	Servo 3

Table 6.6: Home Position of all the servo motors

Joint	Angle
Shoulder	0°
Elbow	30°
Wrist	15°

Motion Sequence

Home
↓
Shoulder
↓
Elbow + Wrist
↓
Hold
↓
Return Home

Experimental Observations

- Accurate positioning
- Smooth movement
- Stable PWM
- Repeatable operation

6.5 Physical Connections

Table 6.7: Servo Motor Connections

Servo Wire	Connection
Red	+6V RPS
Brown	GND
Orange	FPGA JA Pin

DC Motor Connections

FPGA
↓
L293D
↓
Motor

Table 6.8: Power Connections

Device	Supply
FPGA	USB
Servo Motors	6V RPS
DC Motors	6V RPS
L293D	6V RPS

Ground Connections

All grounds were connected together:

- FPGA GND
- Servo GND
- L293D GND
- External Power Supply GND

6.6 FPGA Pin Assignment (XDC)

Table 6.2: Pin Assignment

FPGA Pin	Function
BTN_C	Reset
SW[2:0]	Pose Selection
JA1	Shoulder Servo
JA2	Wrist Servo
JA3	Elbow Servo

XDC Verification

✓ All pins verified in Vivado

✓ No implementation errors

✓ Successful bitstream generation

6.7 External Power Supply Configuration

Table 6.3: Power Requirements

Component	Voltage	Current
FPGA	USB 5V	—
Servo Motor	4.8–6V	Up to 800 mA (stall)
DC Motor	6V	Depends on load
Geared Motor	6V	Higher than DC motor

Supply Configuration

USB
↓
FPGA
(Logic Only)

6V RPS
↓
Servo Motors
↓
DC Motors
↓
L293D

Important Precautions

- Never power servo motors from the FPGA.
- Use a regulated 6V supply.
- Connect all grounds together.
- Verify polarity before powering the circuit.
- Check servo movement before fixing the arm mechanically.

7. EXPERIMENTAL RESULTS AND DISCUSSION

7.1 PWM Verification Using LEDs

Objective

To verify the correctness of PWM signal generation before interfacing motors.

Experimental Procedure

- PWM module implemented in Verilog HDL.
- Design downloaded to Nexys-4 FPGA.
- PWM output connected to onboard LEDs.
- Duty cycle varied using FPGA switches.
- LED brightness observed.

Experimental Results

Table 7.1: LED Brightness Observation

Duty Cycle (%)	LED Brightness
20	Dim (Fig 7.1)
50	Moderate (Fig 7.2)

Duty Cycle (%)	LED Brightness
80	Bright (Fig 7.3)
100	Very Bright (Fig 7.4)

Discussion

The LEDs responded exactly as expected. Increasing the duty cycle increased the average ON time of the LEDs, resulting in higher brightness. This confirmed that the FPGA generated stable PWM signals and validated the PWM module before motor interfacing.

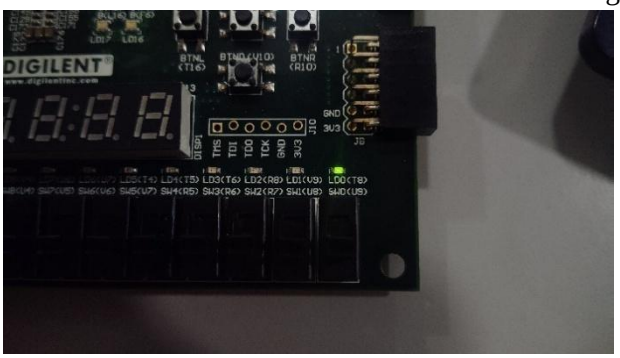

Fig 7.1: LED Brightness for 20% duty cycle at 00

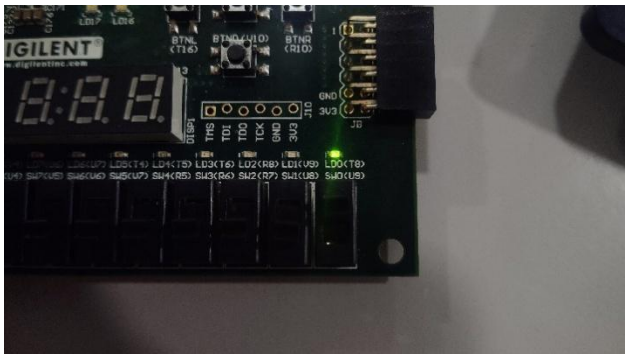

Fig 7.2: LED Brightness for 50% duty cycle at 01

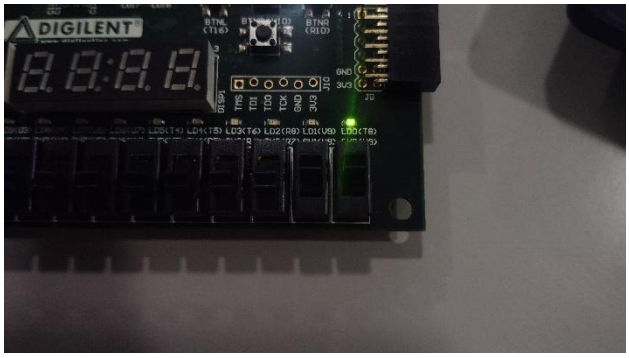

Fig 7.3: LED Brightness for 80% duty cycle at 10

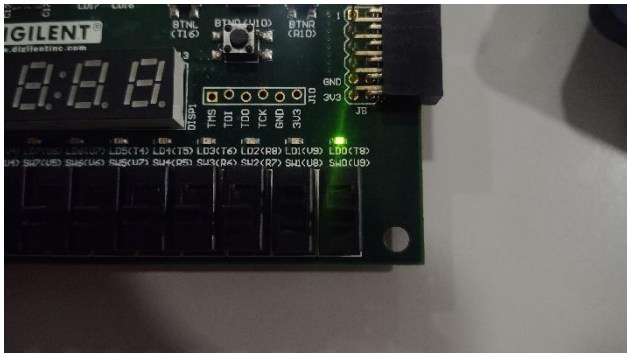

Fig 7.4: LED Brightness for 100% duty cycle at 11

7.2 DC Motor Speed Analysis

Objective

To investigate the relationship between PWM duty cycle and DC motor speed.

Table 7.2: DC Motor Observation

Duty Cycle (%)	Motor Response
20	Slow (Fig 7.5) – 640-750 RPM

50	Moderate (Fig 7.6) – 900-990 RPM
Duty Cycle (%)	**Motor Response**
80	Fast (Fig 7.7) – 1000-1200 RPM
100	Maximum Speed (Fig 7.8) – 1250-1300 RPM

Experimental Observations

- Smooth increase in speed with duty cycle.
- Stable motor rotation.
- Quick response to switch changes.
- No noticeable vibration during operation

Discussion

The DC motor exhibited a nearly proportional increase in speed with increasing PWM duty cycle. Since the average voltage supplied to the motor increases with duty cycle, the motor rotates faster at higher duty cycles. The FPGA generated stable PWM signals throughout the experiment, confirming the suitability of PWM for DC motor speed control.

Fig 7.5: Speed of DC motor in RPM for 20% duty cycle

Fig 7.6: Speed of DC motor in RPM for 50% duty cycle

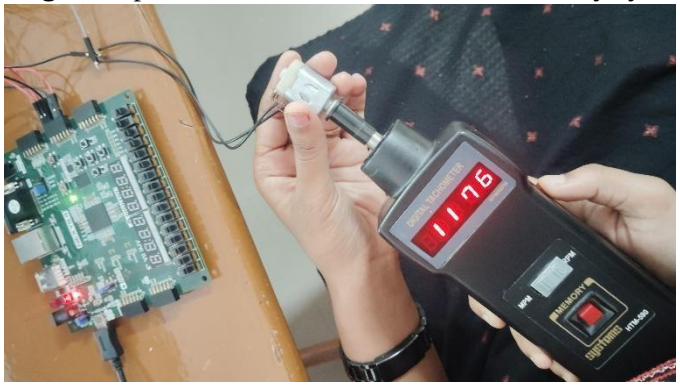

Fig 7.7: Speed of DC motor in RPM for 80% duty cycle

Fig 7.8: Speed of DC motor in RPM for 100% duty cycle

7.3 DC Geared Motor Speed Analysis

Objective

To analyse the effect of PWM duty cycle on a DC geared motor.

Table 7.3: DC Geared Motor Observation

Duty Cycle (%)	**Motor Response**
10	No Rotation
20	No Rotation
31.5	Slow Rotation (Fig 7.9) – 15-25 RPM
50	Stable Rotation (Fig 7.10) – 225-235 RPM
80	High Torque (Fig 7.11) – 250-255 RPM
100	Maximum Speed (Fig 7.12) – 265-270 RPM

Experimental Observations

- Higher startup duty cycle required.
- Higher output torque observed.
- Lower speed than DC motor.
- Stable operation using regulated supply.

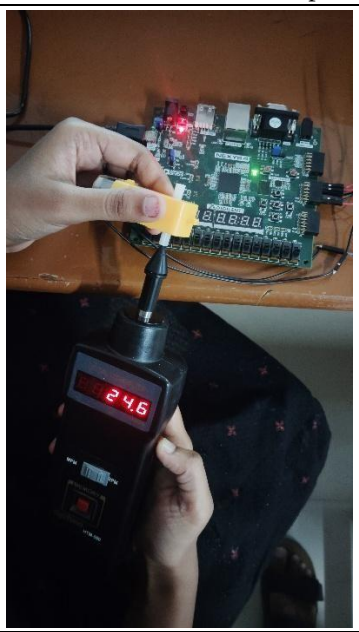

Fig 7.9: Speed of DC Geared motor in RPM for 31.5% duty cycle

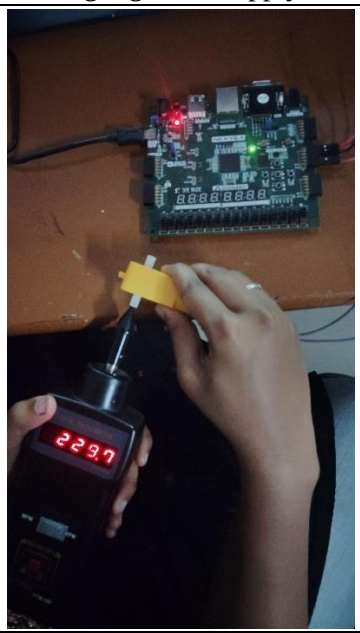

Fig 7.10: Speed of DC Geared motor in RPM for 50% duty cycle

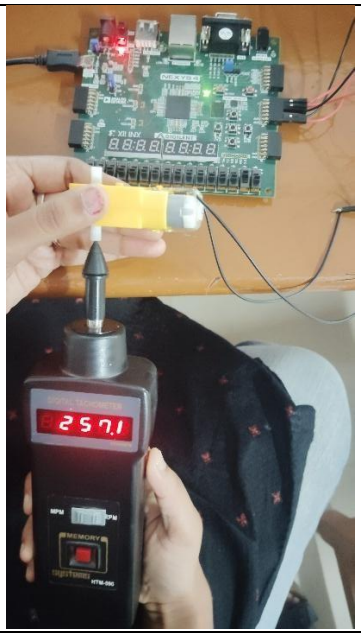

Fig 7.11: Speed of DC Geared motor in RPM for 80% duty cycle

Fig 7.12: Speed of DC Geared motor in RPM for 100% duty cycle

Discussion

Unlike the conventional DC motor, the geared motor required a higher duty cycle before rotation began. The gearbox increased the startup torque requirement while reducing the output speed. Although the geared motor rotated more slowly, it delivered significantly higher torque, making it suitable for heavyload robotic applications.

7.5 Three-Joint Robotic Arm Movement

Table 7.4: Home Position of all the servo motors

Joint	Initial Angle
Shoulder	0°
Elbow	30°
Wrist	15°

Motion Sequence

Home Position
↓
Shoulder Movement
↓
Elbow + Wrist Movement
↓
Hold Position
↓
Return Home

Experimental Results

- Shoulder moved first.
- Elbow and wrist moved simultaneously.
- Smooth coordinated movement.
- Automatic return to home position.

Discussion

The implemented motion sequence closely resembled the movement of a practical robotic manipulator. Independent PWM generation for each servo enabled accurate multi-joint control while maintaining smooth and repeatable operation.

Code

```
`timescale 1ns / 1ps module
servo_controller(        input
clk,     input btnC,

   input [2:0] sw,

   output           servo_shoulder,
output servo_elbow,
   output servo_wrist
);

reg [20:0] shoulder_pw; reg
[20:0] elbow_pw;
reg [20:0] wrist_pw;

reg [31:0] delay_counter;
reg [2:0] state;

//------------------------------------------------
// State Machine
//------------------------------------------------

always @(posedge clk or posedge btnC) begin

   if(btnC)
begin

      state <= 0;
      delay_counter <= 0;

   end
    else
begin

      delay_counter <= delay_counter + 1;

      case(state)

      //------------------------------------------------
      // HOME
      //------------------------------------------------

      3'd0:
begin

         shoulder_pw <= 21'd50000;    // 0°
elbow_pw         <=  21'd75000;       //  30°
```

```
wrist_pw        <=  21'd62500;      //  15°
if(sw  !=  3'b000)                          begin
delay_counter <= 0;             state <= 3'd1;
end

end

      //-----------------------------------------------
      // MOVE SHOULDER
      //-----------------------------------------------

      3'd1:
begin

         case(sw)

            3'b001: shoulder_pw <= 21'd100000; //45°
            3'b010: shoulder_pw <= 21'd150000; //90°
            3'b011:   shoulder_pw   <=   21'd200000;   //135°
3'b100: shoulder_pw <= 21'd250000; //180°
            3'b101:   shoulder_pw   <=   21'd150000;   //90°
3'b110: shoulder_pw <= 21'd100000; //45°
            3'b111: shoulder_pw <= 21'd250000; //180°

         endcase

         if(delay_counter > 32'd50000000)
begin               delay_counter <= 0;
state <= 3'd2;            end

end

      //-----------------------------------------------
      // MOVE ELBOW + WRIST
      //-----------------------------------------------

      3'd2:
begin

         case(sw)

            3'b001:
begin
               elbow_pw <= 21'd125000; //60°
               wrist_pw   <=   21'd100000;   //45°
end
            3'b010:
begin
```

```
                elbow_pw <= 21'd150000; //90°
                wrist_pw   <=   21'd125000;   //60°
end

            3'b011:
begin
                elbow_pw <= 21'd175000; //120°
                wrist_pw   <=   21'd150000;   //90°
end

            3'b100:
begin
                elbow_pw <= 21'd225000; //150°
                wrist_pw   <=   21'd175000;   //120°
end

            3'b101:
begin
                elbow_pw <= 21'd250000; //180°
                wrist_pw   <=   21'd200000;   //135°
end

            3'b110:
begin
                elbow_pw <= 21'd150000; //90°
wrist_pw <= 21'd250000; //180°                end

            3'b111:
begin
                elbow_pw <= 21'd250000; //180°
                wrist_pw   <=   21'd250000;   //180°
end

          endcase

          if(delay_counter > 32'd50000000)
begin               delay_counter <= 0;
state <= 3'd3;            end

end

      //------------------------------------------------
      // HOLD
      //------------------------------------------------          3'd3:         begin

          if(delay_counter > 32'd50000000)
begin               delay_counter <= 0;
state <= 3'd4;            end
```

```
end

        //------------------------------------------------
        // RETURN HOME
        //------------------------------------------------

        3'd4:
begin

            shoulder_pw         <=          21'd50000;
elbow_pw    <= 21'd75000;
            wrist_pw    <= 21'd62500;

            if(delay_counter > 32'd50000000)
begin               delay_counter <= 0;
state <= 3'd0;              end

end

        endcase

    end

end

//------------------------------------------------
// Servo PWM Modules
//------------------------------------------------

servo_pwm shoulder_inst(

    .clk(clk),
    .pulse_width(shoulder_pw),
    .servo_out(servo_shoulder)

);

servo_pwm elbow_inst(
    .clk(clk),
    .pulse_width(elbow_pw),
    .servo_out(servo_elbow)

);

servo_pwm wrist_inst(

    .clk(clk),
```

```
    .pulse_width(wrist_pw),
    .servo_out(servo_wrist)

);

endmodule

`timescale 1ns / 1ps

module servo_pwm(

    input clk,

    input [20:0] pulse_width,

    output reg servo_out

);

reg [20:0] counter;

always @(posedge clk)
begin

    if(counter >= 21'd1999999)
        counter <= 0;

    else                counter  <=
counter + 1;

end

always @(posedge clk)
begin

    if(counter < pulse_width)
servo_out <= 1'b1;     else
servo_out <= 1'b0;

end

endmodule
```

Test Bench

```
`timescale 1ns / 1ps

module servo_controller_tb;
```

```
reg clk; reg
btnC;

reg [2:0] sw;

wire servo_shoulder; wire
servo_elbow;
wire servo_wrist;

servo_controller uut(

    .clk(clk),
    .btnC(btnC),

    .sw(sw),

    .servo_shoulder(servo_shoulder),
    .servo_elbow(servo_elbow),
    .servo_wrist(servo_wrist)

);

always #5 clk = ~clk;

initial begin

    clk = 0;
btnC = 1;     sw
= 3'b000;

    #100;
btnC = 0;

    sw          =          3'b001;
#300000000;

    sw          =          3'b010;
#300000000;

    sw = 3'b011;
    #300000000;

    sw          =          3'b100;
#300000000;

    sw          =          3'b101;
#300000000;
```

```
    sw          =          3'b110;
#300000000;

    sw          =          3'b111;
#300000000;

    sw          =          3'b000;
#300000000;

    $finish;
```

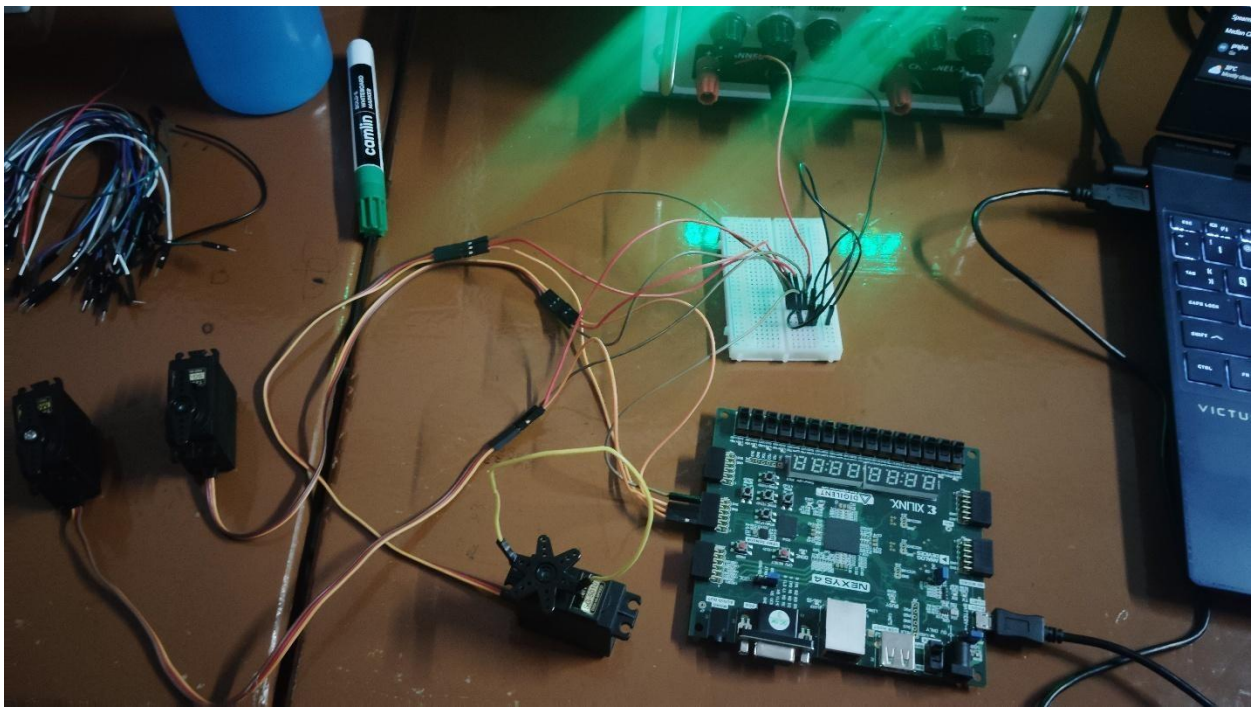

Fig 7.13: Experimental set up of all the servo motors used for 3 joints

7.6 Duty Cycle vs Speed Analysis

Table 7.5: Duty Cycle Comparison

Duty Cycle	DC Motor Speed	Geared Motor Speed

Low	Low	No Rotation
Medium	Moderate	Slow
High	Fast	Moderate
Maximum	Maximum	Maximum

Discussion

The experimental analysis demonstrated that motor speed increased with duty cycle for both motor types. However, the DC geared motor required a higher minimum duty cycle before rotation because of its gearbox and higher starting torque. The conventional DC motor showed faster speed variation, whereas the geared motor provided higher torque at lower speeds.

7.7 Servo Pulse Width vs Angular Position Analysis

Table 7.6: Pulse Width Analysis

Pulse Width	Servo Position
0.5 ms	0°
1.0 ms	45°
1.5 ms	90°
2.0 ms	135°
2.5 ms	180°

Discussion

The experimental results confirmed that the servo motor position depends on the PWM pulse width rather than the duty cycle. Accurate pulse-width calibration enabled precise positioning of all robotic arm joints, resulting in smooth and repeatable movement throughout the experiments.

7.8 Comparison of Experimental Results

Table 7.7: Motor Comparison

Parameter	DC Motor	DC Geared Motor	Servo Motor
Speed Control	Excellent	Good	Not Applicable
Position Control	No	No	Excellent
Torque	Moderate	High	Moderate
Driver Required	Yes	Yes	No
Feedback	No	No	Internal
Robotic Arm Suitability	Low	Medium	High

Key Findings

- FPGA successfully generated stable PWM signals.
- PWM effectively controlled the speed of DC and geared DC motors.
- The DC geared motor required a higher startup duty cycle but delivered greater torque.
- Servo motor calibration enabled accurate 0°–180° position control.
- The FPGA controlled all three servo motors simultaneously without timing conflicts.
- The implemented finite state machine ensured smooth and coordinated multi-joint robotic arm movement.
- The complete hardware implementation validated the effectiveness of FPGA-based motor control for robotic applications.

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This internship focused on designing, implementing, and experimentally validating an FPGA based robotic arm control system using the Nexys 4 FPGA Development Board and Verilog HDL. The project began with the development of a PWM generator, which was first tested using the onboard LEDs to verify that it produced the expected output signals. After successful verification, the PWM signals were used to control both a conventional DC motor and a DC geared motor through an L293D motor driver. These experiments helped evaluate how changes in the PWM duty cycle affected motor speed and torque. After completing the DC motor experiments, the work was extended to SG5010 servo motors for precise position control. The servo pulse widths were experimentally calibrated to achieve accurate and repeatable angular positioning. Using these calibrated values, a three joint robotic arm with shoulder, elbow, and wrist joints was successfully developed. A finite state machine (FSM) based control algorithm was implemented to coordinate the arm's motion. In this sequence, the shoulder joint moved first, followed by the simultaneous movement of the elbow and wrist, before the arm returned to its predefined home position.

The experimental results confirmed that the FPGA generated PWM signals provided stable, repeatable, and deterministic motor control. The project also demonstrated the benefits of FPGA technology, including parallel processing, precise timing, hardware level execution, and the flexibility to implement real time control algorithms. Overall, the internship successfully met its objectives and demonstrated that FPGA based motor control is an effective and practical solution for robotic arm applications.

8.2 Advantages of FPGA-Based Motor Control

The FPGA-based approach adopted in this project offers several advantages over conventional microcontroller-based motor control systems.

Major Advantages

- **High-speed hardware execution** without software execution delays.
- **Parallel processing**, allowing multiple motors to be controlled simultaneously.
- **Highly accurate PWM generation** with precise timing.
- **Deterministic real-time performance**, making the system suitable for robotics.
- **Easy scalability** for adding additional joints or actuators.
- **Flexible hardware design** through Verilog HDL.
- **Reliable operation** with minimal timing jitter.
- **Reduced CPU dependency** since dedicated hardware performs the control tasks.
- **Reconfigurable architecture**, allowing future upgrades without hardware replacement.
- **Suitable for industrial automation**, embedded systems, and robotic applications.

Practical Benefits Observed During the Project •

Stable PWM generation for all experiments.

- Simultaneous control of three servo motors.
- Smooth robotic arm movement.
- Accurate servo positioning after pulse-width calibration.
- Reliable hardware implementation on the Nexys-4 FPGA.

8.3 Project Limitations

Although the developed system successfully demonstrated FPGA-based motor control, certain limitations remain due to the project's scope and available hardware.

Current Limitations

- Position control is open-loop, with no external feedback.
- No encoder was used to verify the actual joint position.
- Servo positioning depends on internal feedback only.
- Motion is limited to predefined switch-selected poses.
- No inverse kinematics or trajectory planning has been implemented.
- The robotic arm cannot detect or avoid obstacles.
- No object recognition or vision-based control is available.
- The DC motor experiments focused primarily on PWM-based speed control without closed-loop speed regulation.
- The system does not compensate for disturbances or varying mechanical loads.
- Communication with external devices is not implemented.

Despite these limitations, the project successfully demonstrates the fundamental concepts of FPGA-based motor control and provides a strong platform for future research.

8.4 Future Scope

The developed robotic arm can be further improved by incorporating advanced control methods, feedback mechanisms, and intelligent sensing technologies. One of the most valuable enhancements would be the implementation of a closed loop control system using rotary encoders to measure the actual position of

each joint. Continuous encoder feedback would enable accurate monitoring of joint movement and improve positioning accuracy, even when the robotic arm operates under varying load conditions.
Another important improvement is the implementation of an FPGA based Proportional Integral Derivative (PID) controller. By continuously comparing the desired joint position with the actual position obtained from the encoders, the PID controller can minimize positioning errors in real time. This approach would reduce overshoot, shorten the settling time, improve disturbance rejection, and produce smoother and more precise robotic arm movements. As a result, the overall performance of the system would be better suited for industrial and high precision applications.
The project can also be extended by adding IoT connectivity for wireless monitoring and control through Wi Fi or cloud-based platforms. Integrating sensors such as ultrasonic sensors, force sensors, and inertial measurement units (IMUs) would improve safety and allow the robotic arm to respond more effectively to changes in its operating environment. Computer vision techniques using cameras and machine learning models could also be incorporated to enable object detection, object tracking, and automated pick and place operations. These capabilities would allow the robotic arm to interact more intelligently with its surroundings.
Another possible enhancement is upgrading the actuator system by replacing the existing servo motors with Brushless DC (BLDC) motors in applications that require higher efficiency, smoother operation, and longer service life. Combining BLDC motors with three phase motor drivers, encoder feedback, and FPGA based PID control would provide higher accuracy, better dynamic performance, and greater reliability for high-speed industrial robotic systems. With these improvements, the current prototype could evolve into a fully autonomous and intelligent robotic platform suitable for advanced manufacturing, research, medical robotics, and Industry 4.0 applications.

REFERENCES

[1] John J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson, 2005 Chapter 1 (pp. 1–18) – Robotics fundamentals; Chapter 7 (pp. 201–229) – Trajectory generation; Chapter 9 (pp. 262–289) – Linear control of manipulators

[2] Muhammad H. Rashid, *Power Electronics: Circuits, Devices, and Applications*, 4th ed., Pearson, 2014. Chapter 5 – DC–DC Converters (PWM control); Chapter 6 – DC–AC Converters; PWM techniques discussed throughout these chapters

[3] Katsuhiko Ogata, *Modern Control Engineering*, 5th ed., Pearson, 2010 Chapter 1 (pp. 1–12) – Introduction to control systems; Chapter 8 (pp. 567–614) – PID Controllers; Chapter 10 (pp. 739–817) – Servo systems and state-space control

[4] A. Z. Mansoor, M. R. Khalil, and O. A. Jasim, "Position Control of DC Servo Motors Using SoftCore Processor on FPGA to Move Robot Arm," *Al-Rafidain Engineering Journal*, vol. 19, no. 4, pp. 74–83, 2011.

[5] Y.-S. Kung, J.-M. Lin, Y.-J. Chen, and H.-H. Chou, "Field Programmable Gate Array-Based Servo Control Integrated Chip for a Six-Axis Articulated Robot Manipulator," *Advances in Mechanical Engineering*, vol. 8, no. 4, pp. 1–13, 2016.

[6] H. Karcı and A. Tangel, "Design and Prototype Implementation of a Five-Degree-of-Freedom Mobile Robot Arm Based on FPGA," *Journal of the Faculty of Engineering and Architecture of Gazi University*, vol. 31, no. 2, pp. 325–336, 2016.

[7] Z. Wan, A. Mahmoud, Y. Wang, and V. Prasanna, "A Survey of FPGA-Based Robotic Computing," *arXiv preprint arXiv:2009.06034*, 2020.

[8] Z. Wan, A. Mahmoud, Y. Wang, and V. Prasanna, "Robotic Computing on FPGAs: Current Progress, Research Challenges, and Opportunities," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 16, no. 2, Article 15, 2023.

[9] A. R. Ajel, H. M. Abdul Abbas, and M. J. Mnati, "Position and Speed Optimization of Servo Motor Control Through FPGA," *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 18, no. 12, pp. 4–19, 2022.